

House Prices - Advanced Regression Techniques

Anna Montoya and DataCanary. House Prices - Advanced Regression Techniques.
<https://kaggle.com/competitions/house-prices-advanced-regression-techniques>,
2016. Kaggle.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.preprocessing import *

from qiskit_machine_learning.algorithms.regressors import VQR
from qiskit_algorithms.optimizers import COBYLA, L_BFGS_B, ADAM, GradientDescent
from qiskit.circuit.library import ZZFeatureMap, TwoLocal, RealAmplitudes
from IPython.display import clear_output
from sklearn.metrics import mean_squared_error, root_mean_squared_error,
from sklearn.model_selection import train_test_split
from qiskit.circuit import Parameter
from qiskit import QuantumCircuit
from qiskit_machine_learning.neural_networks import SamplerQNN, EstimatorQNN
from qiskit_machine_learning.algorithms.regressors import NeuralNetworkRegressor
from qiskit_machine_learning.circuit.library import QNNCircuit
```

```
In [2]: raw_data = pd.read_csv("data/train.csv")
```

```
In [3]: raw_data
```

```
Out[3]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape
0	1	60	RL	65.0	8450	Pave	NaN	Reg
1	2	20	RL	80.0	9600	Pave	NaN	Reg
2	3	60	RL	68.0	11250	Pave	NaN	IR
3	4	70	RL	60.0	9550	Pave	NaN	IR
4	5	60	RL	84.0	14260	Pave	NaN	IR
...	...	...	...	...	...	...	...	...
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg

1460 rows × 81 columns

```
In [ ]: raw_data.info()
```

```
In [5]: raw_data = raw_data.drop(
        columns=["Id"],
        )

raw_data = raw_data.dropna(axis=1)
```

```
In [ ]: raw_data.info()
```

```
In [7]: feature_columns = [
        "MSSubClass", "YearBuilt", "YearRemodAdd",
        # "MSZoning", "LotArea", "Street", "LotShape", "LandContour",
        # "Utilities", "LotConfig", "LandSlope",
        # "Neighborhood", "Condition1", "Condition2", "BldgType", "HouseStyle"
        # "RoofStyle", "RoofMatl",
        # "OverallQual", "OverallCond",
        # "Exterior1st", "Exterior2nd", "ExterQual", "ExterCond", "Foundation"
        # "TotalBsmtSF"
    ]

target_columns = ["SalePrice"]

selected_data = raw_data[ feature_columns + target_columns ]
selected_data.head()
```

```
Out[7]:
```

	MSSubClass	YearBuilt	YearRemodAdd	SalePrice
0	60	2003	2003	208500
1	20	1976	1976	181500
2	60	2001	2002	223500
3	70	1915	1970	140000
4	60	2000	2000	250000

```
In [8]: scalers: dict["str", FunctionTransformer] = {
        "SalePrice": MinMaxScaler(),

        "MSSubClass": MinMaxScaler(),

        "MSZoning": LabelEncoder(),
        "Street": LabelEncoder(),
        "LotShape": LabelEncoder(),
        "LandContour": LabelEncoder(),
        "Utilities": LabelEncoder(),
        "LotConfig": LabelEncoder(),
        "LandSlope": LabelEncoder(),

        "Neighborhood": LabelEncoder(),
        "Condition1": LabelEncoder(),
        "Condition2": LabelEncoder(),
        "BldgType": LabelEncoder(),
        "HouseStyle": LabelEncoder(),
    }
```

```

"RoofStyle": LabelEncoder(),
"RoofMatl": LabelEncoder(),

"Exterior1st": LabelEncoder(),
"Exterior2nd": LabelEncoder(),
"ExterQual": LabelEncoder(),
"ExterCond": LabelEncoder(),
"Foundation": LabelEncoder(),

"LotArea": MinMaxScaler(),
"TotalBsmtSF": MinMaxScaler(),

"YearBuilt": MinMaxScaler(),
"YearRemodAdd": MinMaxScaler(),

"OverallQual": None,
"OverallQual": None,
"OverallCond": None,
}

```

```

In [ ]: # Data processing (scaler/transformer)
for column_name in selected_data.columns:
    scaler = scalers.get(column_name)
    if scaler:
        transform_column = selected_data[[column_name]]
        scaler.fit(transform_column)

        transformed = scaler.transform(transform_column).reshape(len(transformed))
        selected_data[[column_name]] = transformed

```

In [10]: selected_data

Out[10]:

	MSSubClass	YearBuilt	YearRemodAdd	SalePrice
0	0.235294	0.949275	0.883333	0.241078
1	0.000000	0.753623	0.433333	0.203583
2	0.235294	0.934783	0.866667	0.261908
3	0.294118	0.311594	0.333333	0.145952
4	0.235294	0.927536	0.833333	0.298709
...	...	...	...	...
1455	0.235294	0.920290	0.833333	0.194556
1456	0.000000	0.768116	0.633333	0.243161
1457	0.294118	0.500000	0.933333	0.321622
1458	0.000000	0.565217	0.766667	0.148903
1459	0.000000	0.673913	0.250000	0.156367

1460 rows × 4 columns

```
In [11]: selected_x = selected_data[feature_columns]
selected_y = selected_data[target_columns]

train_x, test_x, train_y, test_y = train_test_split(selected_x, selected_y)

# print(train_x.head())
# print(train_y.head())
```

```
In [12]: len(train_x)
```

```
Out[12]: 1022
```

```
In [ ]:
```

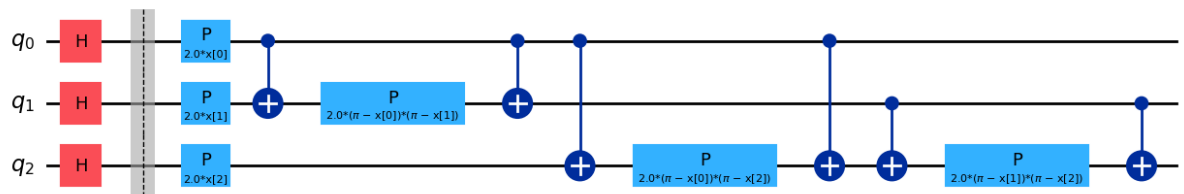
```
In [ ]: plt.plot(selected_data["SalePrice"], selected_data["YearBuilt"], "bo")
plt.xlabel("SalePrice")
plt.show()
```

```
In [ ]: feature_counts = len(selected_data.columns) - 1 # Feature counts = qbits
feature_counts
```

```
In [15]: # Build QNN
```

```
# feature map
feature_map = ZZFeatureMap(feature_dimension=feature_counts, reps=1, insert_barriers=True)
feature_map.decompose().draw(output="mpl")
```

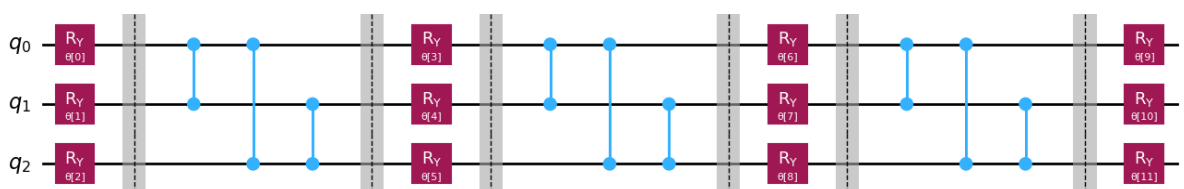
```
Out[15]:
```



```
In [16]: ansatz = TwoLocal(feature_counts, ['ry'], 'cz', reps=3, insert_barriers=True)
ansatz.decompose().draw(output="mpl")
```

```
# Data Reuploading ????
```

```
Out[16]:
```



```
In [ ]: objective_func_vals = []
def callback_graph(weights, obj_func_eval):
    clear_output(wait=True)
    objective_func_vals.append(obj_func_eval)
    plt.title("Objective function value against iteration")
    plt.xlabel("Iteration")
    plt.ylabel("Objective function value")
    plt.plot(range(len(objective_func_vals)), objective_func_vals)
    plt.show()
```

```

# # # # construct a circuit
# qc = QNNCircuit(feature_map=feature_map, ansatz=ansatz)

# # # # construct QNN
# regression_qnn = EstimatorQNN(circuit=qc)

# regressor = NeuralNetworkRegressor(
#     neural_network=regression_qnn,
#     loss="squared_error",
#     optimizer=L_BFGS_B(maxiter=100),
#     callback=callback_graph,
# )

regressor = VQR(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=L_BFGS_B(maxiter=10), # COBYLA(maxiter=1000),
    loss="squared_error",
    callback=callback_graph,
)

```

```
In [ ]: regressor.neural_network._circuit.decompose().draw(output="mpl")
```

```
In [19]: print(regressor.neural_network.num_inputs)
print(regressor.neural_network.num_weights)
```

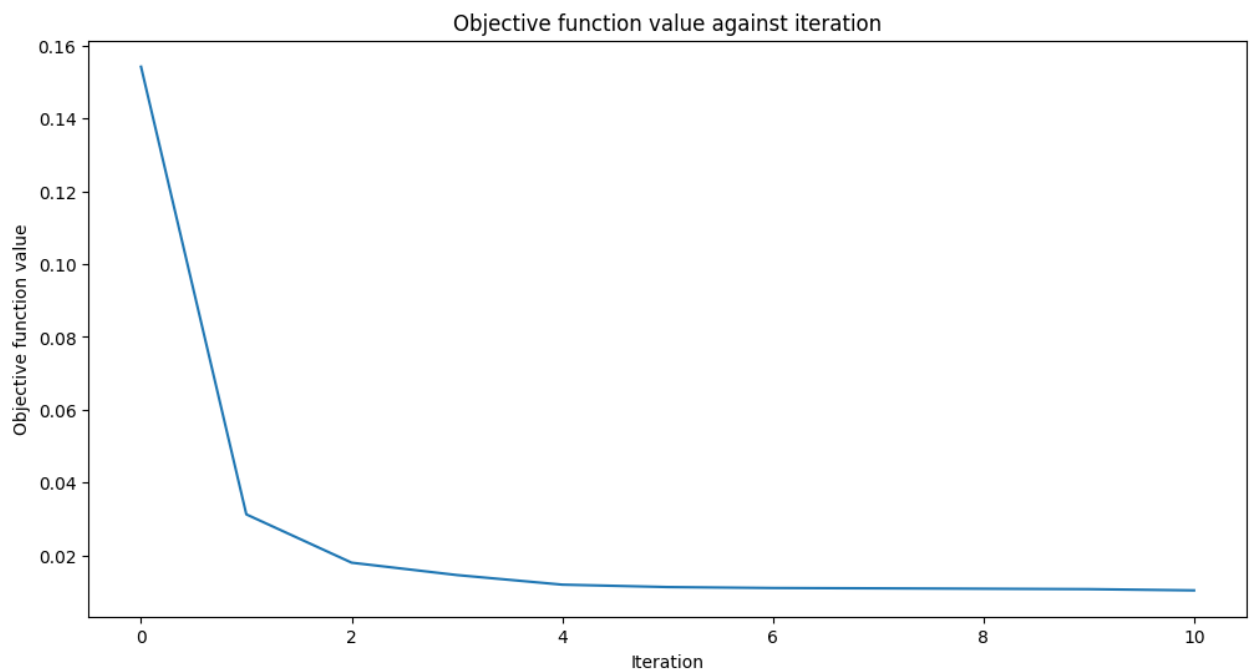
```
3
12
```

```
In [20]: x = np.array(train_x)
y = np.array(train_y)

plt.rcParams["figure.figsize"] = (12, 6)

regressor.fit(x, y)

plt.rcParams["figure.figsize"] = (6, 4)
```



```
In [21]: predict_y = regressor.predict(x)
scaler = scalers["SalePrice"]
predict_y_ = scaler.inverse_transform(predict_y)
y_ = scaler.inverse_transform(y)

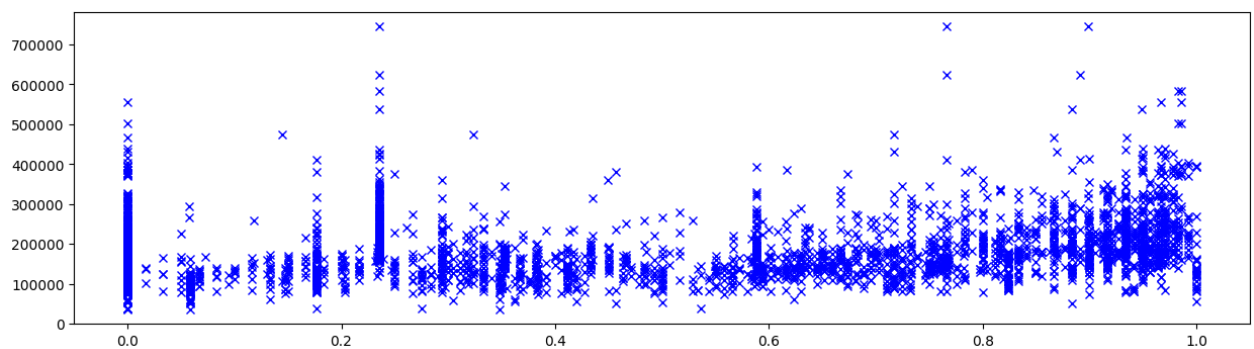
print("RMSE: ", root_mean_squared_error(y_, predict_y_))
print("MSE: ", mean_squared_error(y_, predict_y_))
print("MAE", mean_absolute_error(y_, predict_y_))
print("R^2: ", r2_score(y_, predict_y_))
```

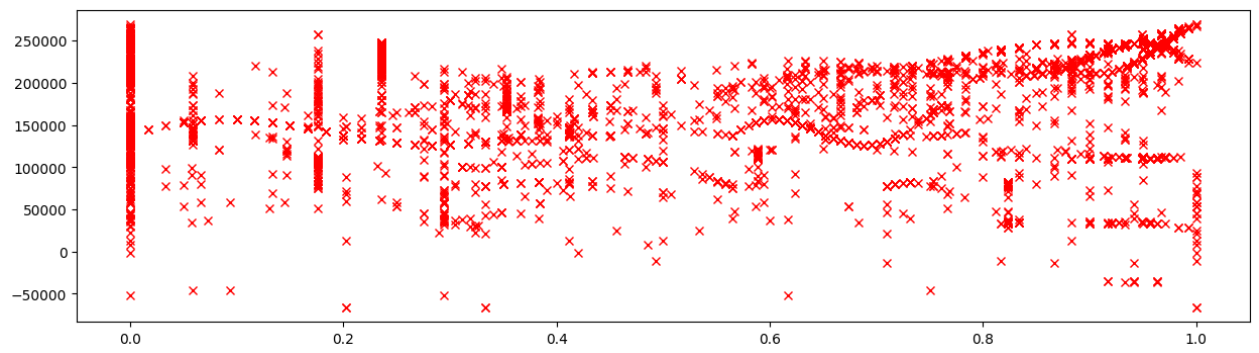
```
RMSE: 73322.3915473206
MSE: 5376173102.218592
MAE 52815.936146833425
R^2: 0.10673661385786426
```

```
In [22]: plt.rcParams["figure.figsize"] = (15,4)
```

```
plt.plot(x, y_, "bx")
plt.show()

plt.plot(x, predict_y_, "rx")
plt.show()
```





In []:

In [23]: `#regressor.save("vqr_f16_v0")`

In [24]: `# import pickle`

`# scalerfile = 'scaler.sav'`
`# pickle.dump(scalers, open(scalerfile, 'wb'))`